

13. Digital Signature Implementation: Implement digital signatures for authentication and data integrity verification

Aim

To implement a **Digital Signature** for authentication and data integrity verification using Python in **Google Colab**.

Objectives

1. Understand the concept of digital signatures.
2. Generate a private and public key pair.
3. Create a digital signature for a message.
4. Verify the signature using the public key.
5. Demonstrate authentication and data integrity.

1. Theory

A **digital signature** is a cryptographic technique used to verify:

- **Authentication:** confirms that the message was signed using the corresponding private key.
- **Integrity:** confirms that the message has not been changed after signing.
- **Non-repudiation:** provides evidence associated with the private-key holder's signature.

Working

Original Message



Hash the Message



Sign Hash using Private Key



Digital Signature



Send Message + Signature



Receiver uses Public Key



Verify Signature



VALID / INVALID

A digital signature is **not the same as encryption**. Its primary purpose is authentication and integrity verification.

2. Software Requirements

Requirement	Details
Platform	Google Colab
Language	Python
Library	<code>cryptography</code>
Internet	Required for installation
Algorithm	RSA + SHA-256

3. Step 1 — Install Required Library

Create a new cell in Google Colab:

```
!pip install cryptography
```

Expected Output

Successfully installed cryptography

4. Step 2 — Import Libraries

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature
```

```
print("Libraries imported successfully.")
```

Output

Libraries imported successfully.

5. Step 3 — Generate Private Key

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
print("Private key generated successfully.")
```

Output

Private key generated successfully.

Explanation

The **private key** is kept secret and is used to create the digital signature.

6. Step 4 — Generate Public Key

```
public_key = private_key.public_key()
```

```
print("Public key generated successfully.")
```

Output

Public key generated successfully.

The public key can be shared with others and is used to **verify** the signature.

7. Step 5 — Create Original Message

```
message = b"This is an authentic and confidential message."
```

```
print("Original Message:")
```

```
print(message.decode())
```

Output

Original Message:

This is an authentic and confidential message.

8. Step 6 — Generate Digital Signature

```
signature = private_key.sign(  
    message,  
    padding.PSS(  
        mgf=padding.MGF1(hashes.SHA256()),  
        salt_length=padding.PSS.MAX_LENGTH  
    ),  
    hashes.SHA256()
```

)

```
print("Digital signature generated successfully.")  
print("Signature length:", len(signature), "bytes")
```

Example Output

Digital signature generated successfully.

Signature length: 256 bytes

The exact signature bytes are not printed because they are binary data.

9. Step 7 — Verify Digital Signature

try:

```
public_key.verify(  
    signature,  
    message,  
    padding.PSS(  
        mgf=padding.MGF1(hashes.SHA256()),  
        salt_length=padding.PSS.MAX_LENGTH  
    ),  
    hashes.SHA256()  
)
```

```
print("Digital Signature: VALID")  
print("Authentication: SUCCESS")
```

```
print("Data Integrity: VERIFIED")
```

```
except InvalidSignature:
```

```
    print("Digital Signature: INVALID")
```

Output

Digital Signature: VALID

Authentication: SUCCESS

Data Integrity: VERIFIED

10. Step 8 — Test Modified Message

Now we will change the message after the signature has been created.

```
modified_message = b"This is a modified message."
```

```
try:
```

```
    public_key.verify(
```

```
        signature,
```

```
        modified_message,
```

```
        padding.PSS(
```

```
            mgf=padding.MGF1(hashes.SHA256()),
```

```
            salt_length=padding.PSS.MAX_LENGTH
```

```
        ),
```

```
        hashes.SHA256()
```

```
    )
```

```
print("Digital Signature: VALID")
```

```
except InvalidSignature:
```

```
    print("Digital Signature: INVALID")
```

```
    print("Data Integrity: FAILED")
```

```
    print("Message has been modified.")
```

Output

Digital Signature: INVALID

Data Integrity: FAILED

Message has been modified.

This demonstrates that modifying the message causes signature verification to fail.

11. Complete Practical Program

You can also run the following **single program** in Google Colab:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
from cryptography.hazmat.primitives import hashes
```

```
from cryptography.exceptions import InvalidSignature
```

```
# -----
```

```
# DIGITAL SIGNATURE IMPLEMENTATION
```

```
# -----
```

```
print("DIGITAL SIGNATURE IMPLEMENTATION")
```

```
print("-----")
```

```
# 1. Generate Private Key
```

```
private_key = rsa.generate_private_key(  
    public_exponent=65537,  
    key_size=2048  
)
```

```
print("\n1. Private Key Generated")
```

```
# 2. Generate Public Key
```

```
public_key = private_key.public_key()
```

```
print("2. Public Key Generated")
```

```
# 3. Create Message
```

```
message = b"This is an authentic message."
```

```
print("3. Original Message:")
```

```
print(message.decode())
```

```
# 4. Create Digital Signature
```

```
signature = private_key.sign(
```

```
message,
padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()),
    salt_length=padding.PSS.MAX_LENGTH
),
hashes.SHA256()
)

print("\n4. Digital Signature Created")
print("Signature Length:", len(signature), "bytes")

# 5. Verify Signature
print("\n5. Verifying Digital Signature...")

try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
```

)

```
print("Signature Status: VALID")
```

```
print("Authentication: SUCCESS")
```

```
print("Data Integrity: VERIFIED")
```

```
except InvalidSignature:
```

```
    print("Signature Status: INVALID")
```

```
# 6. Modify Message
```

```
modified_message = b"This message has been modified."
```

```
print("\n6. Testing Modified Message...")
```

```
try:
```

```
    public_key.verify(
```

```
        signature,
```

```
        modified_message,
```

```
        padding.PSS(
```

```
            mgf=padding.MGF1(hashes.SHA256()),
```

```
            salt_length=padding.PSS.MAX_LENGTH
```

```
        ),
```

```
        hashes.SHA256()
```

)

```
print("Signature Status: VALID")
```

```
except InvalidSignature:
```

```
    print("Signature Status: INVALID")
```

```
    print("Data Integrity: FAILED")
```

```
    print("Modification Detected.")
```

12. Expected Output

DIGITAL SIGNATURE IMPLEMENTATION

1. Private Key Generated

2. Public Key Generated

3. Original Message:

This is an authentic message.

4. Digital Signature Created

Signature Length: 256 bytes

5. Verifying Digital Signature...

Signature Status: VALID

Authentication: SUCCESS

Data Integrity: VERIFIED

6. Testing Modified Message...

Signature Status: INVALID

Data Integrity: FAILED

Modification Detected.

13. Observation Table

No.	Operation	Result
1	Generate private key	Successful
2	Generate public key	Successful
3	Create message	Successful
4	Generate digital signature	Successful
5	Verify original message	Valid
6	Modify message	Detected
7	Verify modified message	Invalid

14. Advantages

1. Provides message authentication.
2. Detects unauthorized modification.
3. Provides data integrity verification.
4. Helps establish trust in electronic communication.
5. Useful for secure digital transactions.
6. Eliminates the need for physical signatures in many digital workflows.

15. Applications

Digital signatures are used in:

- Online banking
- E-commerce
- Digital documents
- E-Governance
- Software distribution
- Electronic contracts
- Secure email
- Online forms
- Certificate-based authentication
- Financial transactions